

Projeto MICROC – Etapa 2

Conversão de uma gramática para LL(1)

2º semestre de 2026

Objetivo

Nesta etapa, o grupo completará os algoritmos que transformam e analisam uma gramática livre de contexto. A entrada fornecida descreve a sintaxe completa da MICROC, mas possui recursão direta à esquerda e alternativas que não podem ser escolhidas com um único token de lookahead.

Resultado da etapa

MicroC.grammar → remoção de recursão direta → fatoração → FIRST, FOLLOW e START → gramática LL(1)

A fatoração e a verificação da condição LL(1) já estão implementadas. O grupo deve completar a remoção de recursão direta à esquerda e o cálculo dos três conjuntos. Esta atividade não implementa um parser nem constrói uma AST.

1 Material inicial e entrega

O starter contém:

- `grammar.py`, com as representações de produção e gramática, a leitura de arquivos, a fatoração e as assinaturas a implementar;
- `MicroC.grammar`, a gramática não LL(1) que deve permanecer sem alterações;
- `runner.py`, que executa todo o fluxo e imprime o resultado;
- `tests/test_grammar.py`, com testes públicos em `pytest`;
- `README.md`, `requirements-dev.txt` e `pyproject.toml`, com instruções e configuração; e
- `.github/workflows/tests.yml`, que executa os testes públicos a cada envio ao GitHub.

O grupo pode criar funções auxiliares em `grammar.py`, mas não deve mudar as assinaturas, os atributos públicos ou o comportamento das partes fornecidas. Não substitua a gramática por uma versão transformada manualmente: os testes privados usarão outras gramáticas para avaliar os algoritmos.

A entrega será feita integralmente no repositório do grupo no GitHub. Arquivos enviados pelo Canvas ou por outro canal não substituem o commit nesse repositório.

Prazo desta etapa

A entrega encerra-se no **domingo, 6 de setembro de 2026, às 23h59**, no horário de Brasília. O commit considerado na correção deve estar no repositório correto do grupo no GitHub até esse horário.

2 Formato da gramática

Cada linha não vazia contém um não terminal, `::=` e uma ou mais alternativas separadas por `|` . Símbolos são separados por espaços. Linhas iniciadas por `#` são comentários, e `ε` deve ocupar sozinho uma alternativa vazia.

```
# exemplo
List ::= List COMMA item | item
Optional ::= value | ε
```

O primeiro lado esquerdo define o símbolo inicial. Todo símbolo que aparece em algum lado esquerdo é não terminal; os demais são terminais. Em `MicroC.grammar`, os terminais possuem exatamente os nomes de `TokenKind` publicados na etapa do lexer, como `KW_INT`, `IDENTIFIER` e `LEFT_PAREN`. `EOF` é o marcador de fim usado nos conjuntos e não precisa aparecer em uma produção.

Internamente, uma produção vazia possui `rhs == ()`. A constante `EPSILON` somente representa `ε` nos conjuntos e na apresentação.

3 Contrato de implementação

3.1 Recursão direta à esquerda

Implemente:

```
Grammar.eliminate_direct_left_recursion(
    nonterminal: str,
) -> bool
```

Para as alternativas

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n,$$

use `_fresh_nonterminal` para criar A' e produza:

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A', \quad A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \varepsilon.$$

O método retorna `True` quando transforma o não terminal e `False` quando nenhuma de suas alternativas possui recursão direta. Preserve a ordem relativa das alternativas. A gramática fornecida possui ao menos uma base para cada caso recursivo.

Escopo

`MicroC.grammar` não possui recursão indireta à esquerda. O algoritmo de substituição para recursão indireta não faz parte desta entrega.

3.2 FIRST

Implemente `first_of_sequence` e `build_first`. FIRST deve ser calculado por propagação iterativa até nenhum conjunto mudar. Para uma sequência $X_1 X_2 \dots X_k$, acrescente os terminais de $\text{FIRST}(X_i)$ enquanto todos os símbolos anteriores forem anuláveis. Acrescente ε somente se todos os símbolos forem anuláveis; a sequência vazia possui FIRST igual a $\{\varepsilon\}$.

`build_first` deve substituir `self.first` por um dicionário que possua exatamente uma entrada para cada não terminal.

3.3 FOLLOW

Implemente `build_follow`. Inicialize todos os conjuntos vazios e inclua EOF em $\text{FOLLOW}(S)$, onde S é o símbolo inicial. Para cada produção, percorra o lado direito da direita para a esquerda mantendo o conjunto de terminais que pode suceder o símbolo atual. Repita todas as produções até atingir um ponto fixo.

FOLLOW é definido somente para não terminais e nunca contém ε . `build_first` será chamado antes de `build_follow` pelo fluxo fornecido.

3.4 START

Implemente `build_start`. Cada chave de `self.start` deve ser o próprio objeto `Production`. Para uma produção $A \rightarrow \beta$:

$$\text{START}(A \rightarrow \beta) = \begin{cases} \text{FIRST}(\beta), & \varepsilon \notin \text{FIRST}(\beta), \\ (\text{FIRST}(\beta) - \{\varepsilon\}) \cup \text{FOLLOW}(A), & \varepsilon \in \text{FIRST}(\beta). \end{cases}$$

O método fornecido `build_sets` chama FIRST, FOLLOW e START nessa ordem.

4 Infraestrutura fornecida

`left_factor` procura o maior prefixo comum, cria não terminais auxiliares e repete a transformação até estabilizar. Essa implementação não deve ser substituída e não faz parte da avaliação dos algoritmos do grupo.

Depois dos conjuntos serem calculados, `l1l_conflicts` compara todas as alternativas de cada não terminal. A gramática é LL(1) exatamente quando os STARTs de quaisquer duas alternativas do mesmo não terminal são disjuntos. O runner usa o fluxo:

```
grammar = Grammar.from_file("MicroC.grammar")
grammar.eliminate_all_direct_left_recursion()
grammar.left_factor()
grammar.build_sets()
conflicts = grammar.l1l_conflicts()
```

Uma implementação correta deve produzir uma lista vazia de conflitos sem depender de valores específicos codificados para a gramática MicroC.

5 Restrições

É permitido usar a biblioteca padrão de Python. Não é permitido usar SLY, PLY, Lark, ANTLR ou biblioteca que remova recursão, calcule FIRST/FOLLOW/START ou decida LL(1) pelo grupo. Também não é permitido:

- alterar `MicroC.grammar` para entregar uma gramática pronta;
- preencher os conjuntos com tabelas ou respostas específicas da MicroC;
- alterar a fatoração ou o verificador para esconder conflitos; ou
- remover testes, workflow ou arquivos obrigatórios.

6 Execução e testes

Prepare o ambiente e execute:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python runner.py MicroC.grammar
```

O starter inicialmente lança `NotImplementedError`. Isso é esperado. Os testes públicos verificam separadamente recursão direta, sequências anuláveis, propagação de FOLLOW, START de produções vazias e a integração da gramática MicroC.

Os testes privados usarão gramáticas pequenas adicionais para verificar ponto fixo, múltiplas alternativas recursivas, bases vazias, cadeias de não terminais anuláveis e ausência de resultados codificados manualmente. Eles respeitarão o escopo de recursão direta estabelecido neste enunciado.

7 Checklist de entrega

Antes do prazo, verifique se:

1. `MicroC.grammar` continua igual ao arquivo fornecido;
2. a remoção atua sobre todos os não terminais e não deixa recursão direta;
3. `FIRST` e `FOLLOW` convergem somente quando nenhum conjunto muda;
4. `FIRST` de uma sequência trata todos os prefixos anuláveis;
5. `FOLLOW` do símbolo inicial contém `EOF` e nenhum `FOLLOW` contém `EPSILON`;
6. `START` usa `FOLLOW` exatamente quando a produção é anulável;
7. `python -m pytest -q` passa integralmente; e
8. o runner termina informando que a gramática é LL(1).